

Template Method

Template Method Pattern

Fixed skeleton, pluggable steps

DEFINITION

Define the skeleton of an algorithm in a base method, deferring some steps to subclasses.

Template Method freezes an algorithm's skeleton in a base method that calls a few overridable steps in a fixed order. Subclasses supply the varying parts — required abstract steps and optional hooks — but can never reorder or skip the skeleton. It is the Hollywood Principle: don't call us, the base calls your steps.

WHY IT MATTERS

When several variants share the same overall flow but differ in a step or two, copying the whole flow into each duplicates the invariant and lets versions drift. Template Method writes the order once in the base and defers only the gaps, killing the duplication and keeping you in control of where extension is allowed. The cost is inheritance coupling — subclasses bind to the base, only one variation axis works well, and base changes are fragile.

— How it works

AbstractClass

Holds the template method — the fixed skeleton calling primitive and hook steps.

primitiveOperation()

An abstract step each subclass must supply; the base only declares it.

ConcreteClass

Overrides the deferred steps (and optional hooks), never the skeleton.

HOW TO APPLY

1. Write the invariant flow once in a base `templateMethod()`, in a fixed order.
2. Declare the varying parts as abstract steps (required) or hooks (optional defaults).
3. Subclass to fill the steps; never let a subclass reorder or skip the skeleton.
4. If two steps vary independently, prefer Strategy — Template Method is one-axis.

LITMUS TEST

Do several variants share one fixed flow but differ in a step or two? If the order is invariant and only steps change, template the skeleton and defer the steps.

— Smell → fix

Two reports duplicate the whole render flow just to change one step:

```
class InvoiceReport {
  render() { return head() + rows() + foot() }
}
class TaxReport {
  render() { return head() + taxRows() + foot() } // dup
}
```

↓ HOIST THE SKELETON, DEFER THE STEPS

```
abstract class Report {
  render(): string { // template, fixed order
    return this.head() + this.body() + this.foot()
  }
  protected head() { return '' } // hook (optional)
  protected abstract body(): string // step (required)
}
class Invoice extends Report { body() { return rows() } }
```

The render order lives once in `Report.render()`; subclasses override only `body()` (and optionally `head()` / `foot()`). Add a `WebReport` by subclassing — the skeleton never changes. If steps must vary on several axes, switch to Strategy.

USE WHEN

- ✓ Frameworks with a fixed flow + custom hooks
- ✓ Shared algorithm, varying parts

AVOID WHEN

- ▲ Steps vary so much the skeleton is meaningless — prefer Strategy

— Trade-offs & pitfalls

PROS	CONS
+ Reuse the invariant flow + Control the extension points	− Inheritance-bound − Fragile base class

COMMON PITFALLS

- ▲ Inheritance coupling: subclasses bind to the base's internals, so a base change can break every subclass (fragile base class).
- ▲ One axis only: two independently-varying steps explode into a subclass matrix — Strategy composes far better.
- ▲ Abstract vs hook: an abstract step is mandatory, a hook has a safe default — pick deliberately, and overridden steps must honor the base's contract (Liskov).

WHERE YOU SEE IT

- ▶ Framework lifecycles: JUnit setUp → test → tearDown; HttpServlet.service() dispatching to doGet / doPost.
- ▶ java.util.AbstractList (get / size abstract; iterator / equals templated on them).
- ▶ A build pipeline, or a sort with a pluggable comparator step.

SEE ALSO

Strategy	the composition alternative — varies a whole algorithm at runtime, no inheritance
Factory Method	a Template Method specialized for creation (the deferred step is "create")
Col	"favor composition over inheritance" — why Strategy often beats Template Method
LSP	overridden steps must honor the base's contract or the templated algorithm misbehaves

— Interview & sources

Template Method vs Strategy?

Same goal — vary part of an algorithm. Template uses inheritance and a fixed base skeleton; Strategy composes the varying part as an object, swappable at runtime.

What's the Hollywood Principle?

"Don't call us, we'll call you" — the base class drives the flow and calls down into your overridden steps; you don't call the skeleton.

Abstract step vs hook?

An abstract step is mandatory — the subclass must implement it. A hook has a default (often empty) and override is optional.

Main drawback?

Inheritance coupling: subclasses are tied to the base, only one variation axis works well, and base changes are fragile.

REMEMBER

The base method fixes the algorithm's skeleton and order; subclasses fill only the varying steps.

SOURCES

GoF — "Design Patterns" (1994): Template Method (behavioral)

Freeman & Robson — "Head First Design Patterns" (2nd ed., 2020): Template Method